SHFTR

NPM PACKAGE · NESTJS 11

@scope/nestjs-panel

Usage Guide

Official handbook for installing, configuring, and extending the SHFTR decorator-driven admin panel — runtime module, generator CLI, and React SPA.

DISTRIBUTION

npm · peer deps only for Nest

STACK

NestJS · React · Vite · ts-morph

AUDIENCE

Backend engineers integrating the panel

COMPANION

README.md on npm / GitHub

Contents

1	3	0
NPM INSTALL	DECORATORS	DB COUPLING

01

Overview

`@scope/nestjs-panel` is a standalone npm package that adds an operator admin UI to any NestJS API. Annotate controllers with `@PanelModule`, `@PanelApi`, and `@PanelAction`; run the bundled generator; serve a branded React SPA from the same HTTP process.

Package provides

Nest module, login endpoint, env config factory, metadata decorators, list/pagination DTOs, HMAC auth, static SPA serving, ts-morph generator, and React template.

Your app provides

Nest 11 runtime, domain controllers & services, `.env` values, branding in bootstrap, and optionally guards on domain routes.

> **Config split:** the package defines `ADMIN_PANEL_*` keys and exports `adminPanelConfig`. Your application supplies values via `ConfigModule` and sets `title` / `theme` in code at bootstrap.

02

Installation

```
npm install @scope/nestjs-panel
```

Ensure peer dependencies are satisfied in your Nest application:

```
@nestjs/common  @nestjs/core  @nestjs/config
@nestjs/platform-express  @nestjs/swagger
class-validator  class-transformer
reflect-metadata  rxjs
```

The generator CLI requires `ts-morph` in your project (devDependency):

```
npm install -D ts-morph
```

Recommended npm script

```
{
  "scripts": {
    "admin:generate": "nestjs-panel generate"
  }
}
```

The `nestjs-panel` binary ships with the package. Run it from your API project root (where `package.json` and controller sources live).

03

Architecture

Internal clean-architecture layers (import only from the package root):

```
@scope/nestjs-panel
├── domain/           AdminPanel model, DI tokens, metadata keys
├── application/      @Panel* decorators, DTOs, HostBootstrapOptions
├── infrastructure/   adminPanelConfig, HMAC auth, setupAdminPanel()
├── presentation/     AdminPanelModule, login controller
└── generator/        CLI + React SPA template (build-time)
```

Runtime flow: ConfigModule → AdminPanelModule → setupAdminPanel() → serve SPA + /admin-panel/config Build-time flow: @Panel* metadata → nestjs-panel generate → manifest → Vite build → dist/

Dependency rule: outer layers depend inward. Consumers import only from `@scope/nestjs-panel`.

04

Quick start

1. `npm install @scope/nestjs-panel`
2. Add `ADMIN_PANEL_*` variables to your `.env`
3. Load `adminPanelConfig` and register `AdminPanelModule.forRootAsync()`
4. Call `setupAdminPanel()` in `main.ts` with branding
5. Annotate feature controllers with `@PanelModule` / `@PanelApi` / `@PanelAction`
6. Run `npm run admin:generate`
7. Start your Nest application

RESOURCE	URL
Admin UI	`http://localhost:<port>/<ADMIN_PANEL_PATH>`
Login	`POST /<apiPrefix>/admin-panel/login`
Runtime branding	`GET /<apiPrefix>/admin-panel/config`

05

Environment variables

Set these in **your application's** `.env`. The package reads them via `adminPanelConfig`.

VARIABLE	DEFAULT	DESCRIPTION
`ADMIN_PANEL_ENABLED`	`true` unless production	Enable panel runtime and auth controller
`ADMIN_PANEL_PATH`	`admin`	URL path segment for the SPA
`ADMIN_PANEL_USERNAME`	`admin`	Operator login username
`ADMIN_PANEL_PASSWORD`	`admin`	Operator login password
`ADMIN_PANEL_SECRET`	`change-me-in-production`	HMAC token signing secret

For the generator, your app should also define:

VARIABLE	USED FOR
`APP_PORT`	SPA build-time API base URL
`APP_API_PREFIX`	SPA build-time API prefix

> **Production:** use strong credentials and secret. Set `ADMIN_PANEL_ENABLED=false` when the panel is not required.

06

NestJS integration

AppModule

```
import { adminPanelConfig, AdminPanelModule } from "@scope/nestjs-panel";

@Module({
  imports: [
    ConfigModule.forRoot({
      load: [appConfig, adminPanelConfig],
    }),
    AdminPanelModule.forRootAsync(),
    // ...your feature modules
  ],
})
export class AppModule {}
```

main.ts — bootstrap & branding

```
import { ADMIN_PANEL, AdminPanel, setupAdminPanel } from "@scope/nestjs-panel";

const adminFromEnv = app.get<AdminPanel>(ADMIN_PANEL);

setupAdminPanel(
  app,
  new AdminPanel({
    enabled: adminFromEnv.enabled,
    path: adminFromEnv.path,
    rootUser: adminFromEnv.rootUser,
    secret: adminFromEnv.secret,
    title: "Admin Panel",
    theme: {
      primaryColor: "#0ea5e9",
      sidebarColor: "#0c4a6e",
      logoIcon: "Shield",
      subtitle: "Control Center",
      radius: "0.75rem",
    },
  }),
  { apiPrefix: appConfig.apiPrefix, port: appConfig.port },
);
```

Operational settings come from env (via DI). Branding is code-only and exposed at runtime through `GET /<apiPrefix>/admin-panel/config`.

Optional `distPath` on `HostBootstrapOptions` overrides where the built SPA is loaded from (default: `<cwd>/admin/dist`).

07

Controller annotations

DECORATOR	TARGET	ROLE
`@PanelModule`	Class	Sidebar entry — title, icon, order, pageType
`@PanelApi`	Method	List/info endpoint — columns, filters, pagination
`@PanelAction`	Method	CRUD action — modal form or confirm dialog

```
import {
  PanelModule, PanelApi, PanelAction,
  ListQueryDto, paginated,
} from "@scope/nestjs-panel";

@PanelModule({ title: "Users", icon: "Users", pageType: "list", order: 10 })
@Controller("users")
export class UsersController {
  @Get()
  @PanelApi({ title: "Users", columns: ["id", "email"], pagination: true })
  findAll(@Query() query: UsersListQueryDto) {
    return paginated(rows, total, query.page ?? 1, query.limit ?? 20);
  }

  @Post()
  @PanelAction({ type: "create", label: "Create", dto: CreateUserDto })
  create(@Body() dto: CreateUserDto) { /* ... */ }
}
```

Page types: `list` · `info` · `dashboard` · `custom`

Only controllers decorated with `@PanelModule` appear in the generated sidebar.

08

Generator CLI

```
nestjs-panel generate
# or: npm run admin:generate
```

```
Scan controllers → write manifest → sync React template → vite build → dist/
```

1. Scans your Nest controllers for `@Panel*` metadata (ts-morph).
2. Writes a manifest consumed by the React SPA.
3. Syncs the bundled SPA template and runs `npm install` + `npm run build`.
4. Output defaults to `admin/dist` relative to your project root.

Run the generator whenever controller metadata changes, and before production builds that ship the admin UI alongside the API.

> If `dist/` is missing at startup, `setupAdminPanel` logs a warning and skips static serving until you run the generator.

09

Public API

Import everything from the package root:

EXPORT	PURPOSE
`AdminPanelModule`	Dynamic Nest module (`forRoot` / `forRootAsync`)
`adminPanelConfig`	`registerAs("adminPanel")` factory for ConfigModule
`setupAdminPanel`	Mount SPA + runtime config endpoint
`AdminPanel` , `ADMIN_PANEL`	Config model and DI token
`PanelModule` , `PanelApi` , `PanelAction`	Metadata decorators
`ListQueryDto` , `paginated`	Shared list/pagination helpers
`HostBootstrapOptions`	`{ apiPrefix, port, distPath? }`

Generator internals, auth service implementation, and template paths are not part of the public API.

10

Auth & security

ENDPOINT	BEHAVIOUR
`POST /admin-panel/login`	Validates operator credentials; returns HMAC Bearer token (24h TTL)
`GET /admin-panel/config`	Public JSON — `title` and `theme`
Your domain routes	Not guarded by default — SPA sends Bearer; add guards in your app if needed

Panel login is independent of your application's user authentication. Treat `ADMIN_PANEL_USERNAME` / `ADMIN_PANEL_PASSWORD` as operator secrets.

`AdminPanelAuthService.verifyToken()` is available for optional custom guards.

11

Peer dependencies

The package does not bundle Nest. Your application must provide:

PACKAGE	VERSION
`@nestjs/common`	^11
`@nestjs/core`	^11
`@nestjs/config`	^4
`@nestjs/platform-express`	^11
`@nestjs/swagger`	^11 (optional, used by DTOs)
`class-validator` / `class-transformer`	^0.14 / ^0.5
`reflect-metadata` / `rxjs`	^0.2 / ^7

Node.js 20+ required.

12

Boundaries

- No database, Redis, or ORM coupling — works with any Nest data layer.
- Does not replace application authentication or authorization.
- Does not auto-guard your CRUD endpoints with panel tokens.
- Branding (`title` , `theme`) is never read from env — configure in bootstrap code.
- Generator output is a build artifact in your project — treat as deploy output, not package source.